1. Which control structure executes code conditionally based on a boolean expression?

(A) if statement
(B) goto
(C) typedef
(D) struct

2. In C, which loop tests the termination condition before executing the loop body?

(A) while loop
(B) do-while loop
(C) repeat-until
(D) post-test loop

3. Which loop guarantees that the body executes at least once regardless of condition?

(A) do-while loop
(B) while loop
(C) for loop
(D) nested loop

4. Which keyword immediately exits from the innermost enclosing loop or switch block?

(A) break
(B) continue
(C) return
(D) goto

5. Which keyword skips the remaining statements in the current loop iteration and advances?

(A) continue
(B) break
(C) exit
(D) skip

6. Which statement allows multi-way branching based on an integral expression in C?

(A) switch
(B) select
(C) choose
(D) branch

7. What happens if a break statement is omitted at the end of a switch case?

(A) Fall-through execution
(B) Compilation error
(C) Program terminates
(D) Case repeats

8. Which optional case in a switch block executes when no other case labels match?

(A) default
(B) else
(C) otherwise
(D) catch

9. Which data type is NOT permitted as the switch expression in ANSI C?

(A) float / double
(B) int
(C) char
(D) short

10. In a for loop: for (init; cond; incr), which expression is executed only once?

(A) init (initialization)
(B) cond (condition)
(C) incr (increment)
(D) body

11. Which symbol is required at the end of a do-while loop statement?

(A) Semicolon (;)
(B) Colon (:)
(C) Closing brace only
(D) Period (.)

12. What is the result of writing: for (;;) { ... } in a C program?

(A) Infinite loop
(B) Syntax error
(C) Executes zero times
(D) Executes once

13. Which statement performs an unconditional jump to a labeled statement in the same function?

(A) goto
(B) jump
(C) branch
(D) throw

14. Which standard library function reads a single character from the standard input?

(A) getchar()
(B) putchar()
(C) getch()
(D) printf()

15. Which standard library function writes a single character to standard output?

(A) putchar()
(B) getchar()
(C) putc()
(D) scanf()

16. What special constant integer value does getchar() return when reaching the end of input?

(A) EOF (-1)
(B) 0
(C) NULL ('\0')
(D) 255

17. Why should the variable receiving getchar() return value be declared as int, not char?

(A) To properly hold EOF (-1)
(B) For faster memory access
(C) To avoid truncation
(D) char is unsigned

18. Which function was completely removed from C11 due to inherent buffer overflow vulnerability?

(A) gets()
(B) puts()
(C) fgets()
(D) getchar()

19. Which safe string input function specifies the maximum buffer size and input stream?

(A) fgets()
(B) gets()
(C) scanf()
(D) getstr()

20. What does puts() automatically append to standard output after printing the string?

(A) Newline character ('\n')
(B) Space (' ')
(C) Null terminator
(D) Carriage return

21. Which printf flag left-justifies text within the specified field width?

(A) - (Minus flag)
(B) + (Plus flag)
(C) 0 (Zero flag)
(D) # (Hash flag)

22. What does the format specifier %10.4f indicate for a floating-point number?

(A) Min width 10, 4 decimals
(B) 10 decimals, 4 width
(C) Multiply by 10.4
(D) Precision 10 digits

23. What does the expression while(1) create in a C control flow construct?

(A) Infinite loop
(B) One-time execution
(C) Zero iterations
(D) Syntax error

24. What does the expression while(0) do when placed as the header of a loop?

(A) Never executes body
(B) Infinite loop
(C) Executes once
(D) Throws exception

25. In an if-else construct, to which if statement does an ambiguous else associate?

(A) Nearest preceding if
(B) First outermost if
(C) Any random if
(D) Throws syntax error

26. How can the dangling-else ambiguity be explicitly resolved by the programmer?

(A) Enclosing with braces {}
(B) Adding semicolons
(C) Indenting correctly
(D) Using comments

27. Which format specifier reads a sequence of characters until whitespace is encountered?

(A) %s
(B) %c
(C) %d
(D) %[^]

28. What is the return value of printf()?

(A) Characters printed
(B) Number of arguments
(C) Always 0
(D) Boolean status

29. What is the return value of scanf()?

(A) Items matched/assigned
(B) Total bytes read
(C) Characters printed
(D) Always 0

30. What does scanf("%d", &x) return if the user enters non-numeric text like 'hello'?

(A) 0
(B) 1
(C) -1 (EOF)
(D) Undefined

31. Which header file is required to use exit() and EXIT_SUCCESS in C?

(A) <stdlib.h>
(B) <stdio.h>
(C) <string.h>
(D) <conio.h>

32. In a for loop, multiple loop control variables can be initialized using which operator?

(A) Comma operator (,)
(B) Semicolon (;)
(C) Colon (:)
(D) Ampersand (&)

33. Which loop structure is preferred when the exact number of iterations is known beforehand?

(A) for loop
(B) while loop
(C) do-while loop
(D) goto loop

34. Which loop structure is preferred when iterating based on a sentinel value or external condition?

(A) while loop
(B) for loop
(C) switch statement
(D) do-while loop

35. What will happen if a semicolon is placed immediately after an if condition: if (x > 0);?

(A) Empty statement executed
(B) Syntax compile error
(C) Condition inverted
(D) Block skipped

36. What will happen if a semicolon is placed immediately after a while condition: while(x < 5);?

(A) Infinite loop if x < 5
(B) Syntax compile error
(C) Loop executes once
(D) Terminates gracefully

37. Can case labels in a switch statement contain variable expressions like case x:?

(A) No (must be constant)
(B) Yes (in modern C)
(C) Yes (if int)
(D) Implementation-defined

38. Can two case labels have identical constant values in the same switch block?

(A) No (duplicate error)
(B) Yes (allowed)
(C) Yes (with warning)
(D) Only if separated

39. Can a switch statement be nested inside another switch statement in C?

(A) Yes
(B) No
(C) Only 2 levels
(D) Deprecated

40. Which escape sequence represents the carriage return character in C?

(A) \r
(B) \n
(C) \b
(D) \t

41. What is the output of: int x = 0; if (x = 5) printf("T"); else printf("F");?

(A) T (x is assigned 5)
(B) F
(C) Compilation error
(D) Nothing

42. Why does if (x = 5) evaluate to true in C?

(A) Assignment yields 5
(B) Equality is assumed
(C) x is initialized to 0
(D) Compiler bug

43. What will be printed: int i = 1; switch(i) { case 1: printf("A"); case 2: printf("B"); default: printf("C"); }?

(A) ABC (fall-through)
(B) A
(C) AB
(D) C

44. What is the purpose of intentionally omitting a break statement in switch cases?

(A) Share common logic
(B) Reduce file size
(C) Speed up execution
(D) Avoid stack overflow

45. How does continue behave inside a for loop compared to inside a while loop?

(A) Advances update expr
(B) Skips update expr
(C) Terminates loop
(D) Restarts loop init

46. What is the output: for (int i = 0; i < 5; i++) { if (i == 2) continue; printf("%d ", i); }?

(A) 0 1 3 4
(B) 0 1 2 3 4
(C) 0 1
(D) 3 4

47. What is the output: for (int i = 0; i < 5; i++) { if (i == 2) break; printf("%d ", i); }?

(A) 0 1
(B) 0 1 2
(C) 0 1 3 4
(D) 2 3 4

48. What does scanf("%[^\n]", str) do when reading user input?

(A) Reads until newline
(B) Reads single word
(C) Reads everything
(D) Skips newline

49. What does the asterisk (*) in scanf("%*d %d", &val) indicate?

(A) Suppresses assignment
(B) Pointer dereference
(C) Multiplies input
(D) Matches wildcard

50. What is the output: int a = 5; do { printf("%d ", a); a++; } while (a < 5);?

(A) 5 (executes once)
(B) 5 6
(C) Nothing
(D) Infinite 5

51. What does printf("%+d", 42); display?

(A) +42
(B) 42
(C) 42
(D) Error

52. What does printf("%+d", -42); display?

(A) -42
(B) +-42
(C) 42
(D) Error

53. What is the result of: int c = 10; while (c --> 0) { } printf("%d", c);?

(A) -1
(B) 0
(C) 1
(D) 10

54. How is the 'goes-to' operator (-->) parsed by the C lexer?

(A) c-- > 0 (postfix dec)
(B) c - - > 0
(C) Special pointer
(D) Syntax error

55. Which scanset in scanf reads ONLY uppercase alphabetic characters?

(A) %[A-Z]
(B) %[a-z]
(C) %[0-9]
(D) %[^A-Z]

56. Which scanset reads until the first semicolon is encountered in the stream?

(A) %[^;]
(B) %[;]
(C) %s
(D) %c

57. What happens when fgets() reads a newline character ('\n') before hitting buffer limit?

(A) Stores '\n' in buffer
(B) Discards '\n'
(C) Replaces with '\0'
(D) Returns EOF

58. How can a trailing newline stored by fgets(buf, sizeof(buf), stdin) be cleanly stripped?

(A) buf[strcspn(buf,"\n")]=0
(B) buf[strlen(buf)]=0
(C) buf - 1 = '\0'
(D) strip(buf)

59. What is the output: int x = 3; switch(x) { default: printf("D"); case 1: printf("1"); break; }?

(A) D1 (fall-through)
(B) D
(C) 1
(D) Compilation error

60. Can the default case be positioned before case labels in a switch statement in C?

(A) Yes (syntactically legal)
(B) No (must be at end)
(C) Only if break present
(D) Throws warning

61. In nested loops, what does a break statement exit from?

(A) Innermost loop only
(B) All nested loops
(C) Outermost loop
(D) Current iteration only

62. How can a programmer cleanly break out of multiple nested loops simultaneously in C?

(A) Use goto to outer label
(B) Use double break
(C) Use return only
(D) Impossible in C

63. Which standard library function terminates the entire calling process immediately?

(A) exit()
(B) break
(C) return
(D) abort()

64. What is the difference between exit(0) and abort() in C?

(A) abort skips cleanups
(B) exit abnormal term
(C) abort is normal exit
(D) Identical behavior

65. What is the value of x after: int x = 0; for (int i = 0; i < 3; i++) for (int j = 0; j < 2; j++) x++;?

(A) 6
(B) 5
(C) 9
(D) 3

66. What is the mathematical time complexity of nested loops of size N and M: O(N * M)?

(A) O(N * M)
(B) O(N + M)
(C) O(N^2)
(D) O(log(N*M))

67. Which I/O function reads unbuffered, non-echoed keystrokes in Windows conio.h?

(A) _getch()
(B) getchar()
(C) scanf()
(D) fgetc()

68. Why is conio.h (<getch>) non-standard and avoided in portable POSIX C programming?

(A) Not part of ANSI/ISO C
(B) Security violation
(C) Slow performance
(D) Requires C++

69. In Euclidean algorithm for GCD, which loop condition determines the termination?

(A) b != 0 (while b != 0)
(B) a == b
(C) a == 0
(D) b == 1

70. What is the formula to compute Least Common Multiple (LCM) from GCD of two positive integers a, b?

(A) (a * b) / GCD(a, b)
(B) a + b - GCD(a, b)
(C) GCD(a, b) / (a * b)
(D) a * GCD(a, b)

71. Which control structure is ideal for categorizing biological DNA nucleotides (A, C, G, T)?

(A) switch(base)
(B) nested while
(C) goto ladder
(D) ternary operator

72. What does the format specifier %02X produce when printing the byte value 10?

(A) "0A"
(B) "10"
(C) "A"
(D) "0010"

73. What does printf("%*d", width, val); allow the programmer to do dynamically?

(A) Specify width via arg
(B) Print wildcard char
(C) Dereference pointer
(D) Format binary

74. What does printf("%.*f", precision, val); allow the programmer to do?

(A) Specify precision via arg
(B) Print raw pointer
(C) Round to nearest int
(D) Scientific format

75. What is the danger of writing while(count < 10); count++; with an accidental semicolon?

(A) Infinite loop (count unupdated)
(B) count increments to 10
(C) Compilation error
(D) count skips loop

76. What is the mathematical output of Armstrong number algorithm for 371?

(A) 3^3 + 7^3 + 1^3 == 371
(B) 3*7*1 == 21
(C) Prime number
(D) Fibonacci term

77. In verifying whether integer N is prime, what is the optimal upper limit for trial division?

(A) sqrt(N)
(B) N / 2
(C) N - 1
(D) log2(N)

78. Why does trial division up to sqrt(N) suffice to prove primality of N?

(A) Factors occur in pairs
(B) Odd numbers only
(C) Fermat's little theorem
(D) Euler's totient

79. In an iterative Fibonacci generator: t = a + b; a = b; b = t; what are the initial terms?

(A) a = 0, b = 1
(B) a = 1, b = 1
(C) a = 0, b = 0
(D) a = 2, b = 3

80. What is the 10th Fibonacci number in the standard sequence (0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55)?

(A) 55 (if 0-indexed: 34)
(B) 34
(C) 89
(D) 21

81. In computing Taylor series for e^x: sum += term; term *= x / i; what guarantees termination?

(A) term < EPSILON (tolerance)
(B) i == 1000
(C) term == 0
(D) sum overflows

82. What causes catastrophic cancellation in quadratic root formula (-b + sqrt(D)) / (2a) when b^2 >> 4ac?

(A) Subtracting nearly equal nos
(B) Division by zero
(C) Underflow in ac
(D) Integer truncation

83. How does the alternative numerically stable quadratic formula compute the first root x1?

(A) (-b - sgn(b)*sqrt(D))/2a
(B) 2c / (-b - sqrt(D))
(C) -b / 2a
(D) sqrt(D) / 2a

84. What does Duff's Device exploit in ANSI C grammar to achieve loop unrolling?

(A) Interleaving switch & while
(B) Recursive macros
(C) Inline assembly
(D) Goto tables

85. In Duff's Device: switch(count % 8) { case 0: do { *to = *from++; case 7: ... } while(--n > 0); } is this legal?

(A) Yes (valid ANSI C)
(B) No (switch inside loop)
(C) Syntax error
(D) Compiler dependent

86. In bioinformatics, which nucleotide triplet represents the universal eukaryotic Start Codon?

(A) ATG (Methionine)
(B) TGA
(C) TAA
(D) TAG

87. Which triplet combinations represent translation Stop Codons in DNA reading frames?

(A) TAA, TAG, TGA
(B) ATG, ACG, AGG
(C) CAA, CAG, CGA
(D) TAT, TAC, TAA

88. How many reading frames must be checked when translating double-stranded DNA into protein?

(A) 6 reading frames
(B) 3 reading frames
(C) 1 reading frame
(D) 2 reading frames

89. In calculating oligonucleotide melting temperature (Wallace rule): Tm = 2*(A+T) + 4*(G+C), what is Tm for 5'-ATGCG-3'?

(A) 16 deg C (2*2 + 4*3)
(B) 14 deg C
(C) 18 deg C
(D) 20 deg C

90. Why is Wallace's rule (2*(A+T) + 4*(G+C)) only valid for short primers (14-20 nt)?

(A) Neglects nearest-neighbor
(B) Overestimates GC bonds
(C) Assumes RNA duplexes
(D) Ignores magnesium ions

91. What security vulnerability arises from calling printf(user_buffer) without format specifier?

(A) Format string attack
(B) Heap corruption
(C) Integer overflow
(D) Infinite recursion

92. How can a format string vulnerability be completely neutralized in C?

(A) printf("%s", user_buffer)
(B) puts(user_buffer)
(C) Both A and B are secure
(D) printf(user_buffer, "")

93. What does scanf("%20s", buffer) prevent compared to scanf("%s", buffer)?

(A) Buffer overflow vulnerability
(B) Reading whitespace
(C) Reading numbers
(D) Slow file I/O

94. When reading FASTA file header lines starting with '>', which control logic identifies headers?

(A) if (line[0] == '>')
(B) if (strstr(line, "DNA"))
(C) switch(strlen(line))
(D) while(line != EOF)

95. In C, what is the result of evaluating: int x = 1; switch(x) { case 1: { int y = 20; printf("%d", y); } break; }?

(A) Prints 20 (block scope)
(B) Compilation error
(C) y is invisible
(D) Undefined behavior

96. Why must variable declarations directly inside switch case labels be enclosed in braces {}?

(A) Prevents jumping over init
(B) ANSI C forbids local vars
(C) Saves stack memory
(D) Enforces static storage

97. What is the return value of feof(FILE *stream) when the end of stream has NOT yet been reached?

(A) 0 (zero)
(B) Non-zero (true)
(C) -1 (EOF)
(D) NULL

98. Why is while (!feof(fp)) considered a dangerous anti-pattern in C file and stream processing?

(A) feof true only AFTER read fails
(B) feof skips first byte
(C) feof closes stream
(D) feof triggers EOF trap

99. What is the correct robust pattern for reading lines from a stream until end-of-file?

(A) while (fgets(...) != NULL)
(B) while (!feof(stdin))
(C) while (feof == 0)
(D) while (read() > 0)

100. What does fflush(stdout) accomplish after a series of unformatted printf calls?

(A) Forces buffer write to screen
(B) Clears input stream buffer
(C) Resets file pointer to 0
(D) Deallocates stdout buffer

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain conditional branching constructs in C: if, if-else, nested if, and else-if ladders with execution flowcharts.

Q2. Analyze the Dangling-Else Problem in C, detailing compiler resolution rules and defensive coding standards.

Q3. Detail the switch-case statement, explaining integral expression constraints, fall-through mechanics, and jump tables.

Q4. Compare while, do-while, and for loops in C, contrasting entry-controlled vs exit-controlled evaluation with practical use-cases.

Q5. Explain Jump Statements in C (break, continue, goto, return), detailing execution semantics and structured programming principles.

Q6. Detail formatted console output using printf(), explaining field width, precision modifiers, alignment flags, and return values.

Q7. Detail formatted console input using scanf(), explaining address-of operator, whitespace skipping, scansets, and return values.

Q8. Compare character-level I/O functions: getchar(), putchar(), getch(), and getche(), explaining terminal buffering modes.

Q9. Analyze why gets() is dangerously vulnerable to Buffer Overflow and explain fgets() and puts() safe alternatives.

Q10. Write a complete ANSI C program to test whether a given positive integer is prime using optimized trial division up to sqrt(N).

Q11. Write an ANSI C program to generate the Fibonacci sequence up to N terms, discussing iterative space-time complexity.

Q12. Write an ANSI C program to compute the Greatest Common Divisor (GCD) using the Euclidean Algorithm and find the LCM.

Q13. Write an ANSI C program to check whether a given integer is a Palindrome Number by reversing its digits.

Q14. Write an ANSI C program to verify whether a given positive integer is an Armstrong Number (Narcissistic Number).

Q15. Write an ANSI C program to compute the Taylor series expansion of e^x to a specified convergence tolerance epsilon.

Q16. Write an ANSI C program to compute the roots of a quadratic equation ax^2 + bx + c = 0, handling real, equal, and complex roots.

Q17. Write an ANSI C program using switch-case to count nucleotide frequencies (A, C, G, T) and calculate GC content percentage.

Q18. Write an ANSI C program to search a DNA sequence for Start Codon (ATG) and identify corresponding Stop Codons in the open reading frame.

Q19. Explain the mechanics and application of the comma operator inside a for loop header for multiple initializations and updates.

Q20. Write an ANSI C program to print Pascal's Triangle up to N rows using nested for loops and combinatorial logic.

Q21. Detail the concept of an infinite loop in C, explaining intentional daemon loops, termination conditions, and accidental causes.

Q22. Write an ANSI C program to calculate the sum of digits of a positive integer until a single digit root is obtained (digital root).

Q23. Write an ANSI C program to convert a decimal integer to binary, octal, and hexadecimal bases using iteration and bitwise operators.

Q24. Explain the format string vulnerability in C, showing how user-controlled format strings cause arbitrary memory disclosure and writes.

Q25. Write an ANSI C program to calculate the melting temperature (Tm) of a DNA primer using the Wallace Rule and Salt-Adjusted Formula.

Q26. Explain Duff's Device in C, detailing how interleaved switch and loop statements optimize memory block copies.

Q27. Write an ANSI C program to reverse an integer array using a while loop and pointers without auxiliary arrays.

Q28. Write an ANSI C program to compute the factorial of an integer iteratively and analyze integer overflow boundaries.

Q29. Explain the dangling-pointer and memory leak traps that occur when managing dynamic loops and jump statements.

Q30. Write an ANSI C program to simulate a bioreactor pH and temperature feedback control loop with safety alarm thresholds.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[C]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[A]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS

if-else, nested if, and else-if ladders with execution flowcharts.

```
return 0;
}
```

Solution to Question 2: Analyze the Dangling-Else Problem in C, detailing compiler resolution rules and defensive coding standards.

```
  else
  printf("Error: Enzyme inactive!\n"); /* NEVER PRINTS! */

  return 0;
  }
```

Solution to Question 3:

>>**Solution to Question 3:** Detail the switch-case statement, explaining integral expression constraints, fall-through mechanics, and jump tables.

```
  printf("Uracil (Pyrimidine): RNA transcript pair\n");
  break;
  default:
  printf("Unknown or degenerate nucleotide code: %c\n", base);
  break;
  }
  }

  int main(void) {
  process_nucleotide('G');
  process_nucleotide('t');
  process_nucleotide('X');
  return 0;
  }
```

>> Solution to Question 04:

5. Demonstrative Code (Menu Prompt using do-while):

```
#include <stdio.h>

int main(void) {
int option;
do {
printf("\n=== Bioreactor Controller ===\n");
printf("1. Read Dissolved O2\n");
printf("2. Read Impeller RPM\n");
printf("3. Terminate Simulation\n");
printf("Enter choice (1-3): ");
if (scanf("%d", &option) != 1) {
int c; while ((c = getchar()) != '\n' && c != EOF);
option = 0;
continue;
}
} while (option < 1 || option > 3);

printf("Selected option: %d\n", option);
return 0;
}
```

>>Solution to Question 5: Explain Jump Statements in C (break, continue, goto, return), detailing execution semantics and structured programming principles.

- Legitimate Professional Use-Case in Systems C (Linux Kernel Style):
Centralized multi-resource error cleanup in functions to avoid heavily indented nested if statements:

```
#include <stdio.h>
#include <stdlib.h>

int initialize_system(int fail_step) {
int *resA = NULL, *resB = NULL;

resA = (int *)malloc(100 * sizeof(int));
if (!resA) goto fail_alloc_a;

if (fail_step == 1) goto fail_stage_1;

resB = (int *)malloc(200 * sizeof(int));
if (!resB) goto fail_alloc_b;

printf("All resources acquired successfully.\n");
free(resB);
free(resA);
return 0;

fail_alloc_b:
fail_stage_1:
printf("Rolling back: freeing Resource A...\n");
free(resA);
fail_alloc_a:
printf("Initialization failed.\n");
return -1;
}

int main(void) {
initialize_system(1);
return 0;
}
```

>>Solution to Question 6: Detail formatted console output using printf(), explaining field width, precision modifiers, alignment flags, and return values.

- ' ' (space) : Inserts a space before positive numbers to align with negative signs.
- '0' : Pads the field with leading zeros instead of spaces.
- '#' : Alternate format: 0x for %x, 0 for %o, forces decimal point for %f.

4. Dynamic Field Width and Precision (* modifier):
Instead of hardcoding width/precision, pass them as integer arguments via '*':

```
int w = 12, p = 3;
double val = 45.6789;
printf("%*.*f\n", w, p, val); /* Reserves 12 columns with 3 decimals */
```

5. Comprehensive Demonstration Program:

```
#include <stdio.h>

int main(void) {
int id = 42;
double concentration = 0.008456;
const char *gene = "BRCA1";

printf("=== Precision Laboratory Assay Table ===\n");
/* Column headers: Left aligned text, right aligned numbers */
printf("%-10s | %8s | %14s\n", "Gene Target", "Sample ID", "Concentration (M)");
printf("-----------------------------------------------\n");

/* Formatted data rows */
printf("%-10s | %08d | %14.6f\n", gene, id, concentration);
printf("%-10s | %08d | %14.3e\n", "TP53", 105, 0.00000123);
printf("%-10s | %+8d | %14.2f\n", "EGFR", -14, 125.4);

/* Character count return verification */
int chars_written = printf("Formatted log line written.\n");
printf("Prior line character count: %d\n", chars_written);

return 0;
}
```

>> **Solution to Question 7:** Detail formatted console input using scanf(), explaining address-of operator, whitespace skipping, scansets, and return values.

newlines).
- Exception: %c and %[] (scansets) DO NOT skip leading whitespace! They read the immediate next character, which is frequently the leftover newline ('\n') from previous input.
- Fix: Use a leading space in format string: scanf(" %c", &ch);

4. Advanced Scansets (%[...] and %[^...]):
- Positive Scanset: %[A-Za-z] reads only matching alphabetic characters.
- Negated Scanset: %[^\n] reads everything up to the next newline (reads entire lines including spaces).
- Assignment Suppression (*): %*d reads and discards an integer without assigning to a variable.

5. Demonstration Program:

```
#include <stdio.h>

int main(void) {
char header[64];
char dna[128];
int read_length;

printf("Enter FASTA header (e.g. >Seq1_Human): ");
/* Read header including spaces until newline */
if (scanf(" %[^\n]", header) != 1) return 1;

printf("Enter DNA sequence bases (A, C, G, T only): ");
/* Scanset: read only valid nucleotide characters */
if (scanf(" %[ACGTacgt]", dna) != 1) {
printf("Error: Non-nucleotide character detected!\n");
return 1;
}

printf("Enter read length: ");
if (scanf("%d", &read_length) != 1) {
printf("Error: Numeric read length expected!\n");
return 1;
}

printf("\n--- Successfully Parsed Record ---\n");
printf("Header: %s\nSequence: %s\nLength: %d bp\n", header, dna, read_length);

return 0;
}
```

>> Solution to Question 8: Compare character-level I/O functions: getchar(), putchar(), getch(), and getche(), explaining terminal buffering modes.

- Echo vs Non-Echo: Whether entered keystrokes are printed automatically onto the terminal screen.

2. Comparison of Standard and Conio Functions:
Function Standard Buffering Screen Echo Header File
getchar() ANSI / ISO C Line-Buffered Echoed <stdio.h>
putchar() ANSI / ISO C Line-Buffered Outputs char <stdio.h>
getch() Non-Standard Unbuffered (Raw) NO Echo <conio.h> (DOS/Win)
getche() Non-Standard Unbuffered (Raw) Echoed <conio.h> (DOS/Win)

3. Detailed Analysis of getchar():
- Prototype: int getchar(void);
- Return Value: Returns character as unsigned char cast to int, or EOF (-1) upon reaching end-of-file.
- Crucial Rule: Always declare receiving variable as int:
int ch; /* NOT char ch! */
while ((ch = getchar()) != EOF) { ... }
If char is signed 8-bit, reading byte 0xFF would accidentally match EOF (-1)!

4. Complete Filtering Program:
```
#include <stdio.h>
#include <ctype.h>

int main(void) {
int ch;
int base_count = 0;

printf("Enter raw sequence stream (press Ctrl+Z on Windows / Ctrl+D on Linux to end):\n");

while ((ch = getchar()) != EOF) {
/* Filter and clean DNA sequence stream */
char upper = (char)toupper(ch);
if (upper == 'A' || upper == 'C' || upper == 'G' || upper == 'T') {
putchar(upper);
base_count++;
if (base_count % 50 == 0) putchar('\n'); /* Format 50 bp per line */
}
}
putchar('\n');
printf("Total valid bases processed: %d\n", base_count);

return 0;
}
```

>> **Solution to Question 9:** Analyze why gets() is dangerously vulnerable to Buffer Overflow and explain fgets() and puts() safe alternatives.

- Exploitation (Buffer Overflow): If user inputs 500 bytes into a 64-byte stack buffer, input overruns the buffer boundaries, corrupting adjacent stack variables, frame pointers, and the function's Return Address!
- Attack Vector: Attackers inject malicious machine shellcode and hijack instruction pointer (EIP/RIP), gaining root/arbitrary execution.
- Historical Standard Deprecation: Deprecated in ISO C99; COMPLETELY REMOVED from the ISO C11 standard.

2. The Safe Alternative: fgets():
- Prototype: char *fgets(char *str, int num, FILE *stream);
- Safety: Takes explicit buffer size 'num'. Reads at most (num - 1) characters, appending terminating null byte '\0'.
- Buffer Handling: If room exists, fgets() stores the trailing '\n'.
- Removing Trailing Newline:
str[strcspn(str, "\n")] = '\0';

3. Safe Demonstration Program:

```
#include <stdio.h>
#include <string.h>

#define MAX_BUFFER 64

int main(void) {
char sample_name[MAX_BUFFER];

printf("Enter clinical sample identifier (safe fgets): ");
if (fgets(sample_name, sizeof(sample_name), stdin) != NULL) {
/* Strip trailing newline character */
sample_name[strcspn(sample_name, "\r\n")] = '\0';

printf("Successfully captured: \"%s\" (Length: %zu chars)\n",
sample_name, strlen(sample_name));
puts("Record saved securely without buffer overflow risk.");
}

return 0;
}
```

>> **Solution to Question 10:** Here is a complete ANSI C program to test whether a given positive integer is prime using optimized trial division up to sqrt(N),

2. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdbool.h>
#include <math.h>

bool is_prime(long long n) {
if (n <= 1) return false;
if (n == 2 || n == 3) return true;
if (n % 2 == 0 || n % 3 == 0) return false;

/* 6k +/- 1 optimization up to sqrt(n) */
long long limit = (long long)sqrt((double)n);
for (long long i = 5; i <= limit; i += 6) {
if (n % i == 0 || n % (i + 2) == 0) {
return false;
}
}
return true;
}

int main(void) {
long long test_cases[] = {1, 2, 17, 49, 97, 1000000007LL, 1000000009LL};
int count = sizeof(test_cases) / sizeof(test_cases[0]);

printf("=== High Performance Primality Verification ===\n");
for (int i = 0; i < count; i++) {
long long val = test_cases[i];
printf("Number %12lld is %s\n", val, is_prime(val) ? "PRIME" : "COMPOSITE");
}
return 0;
}
```

>>P Solution to Question 11: an ANSI C program to generate the Fibonacci sequence up to N terms, discussing iterative space-time complexity.

```
void generate_fibonacci(int n) {
if (n <= 0) {
printf("Sequence length must be positive.\n");
return;
}

unsigned long long a = 0, b = 1;

printf("Fibonacci Sequence (%d terms):\n", n);
for (int i = 0; i < n; i++) {
if (i == 0) {
printf("Term 1: %llu\n", a);
} else if (i == 1) {
printf("Term 2: %llu\n", b);
} else {
unsigned long long next = a + b;
printf("Term %d: %llu\n", i + 1, next);
a = b;
b = next;
}
}
}

int main(void) {
int terms = 15;
generate_fibonacci(terms);
return 0;
}
```

Solution to Question 12: Write an ANSI C program to compute the Greatest Common Divisor (GCD) using the Euclidean Algorithm and find the LCM.

```
long long compute_lcm(long long a, long long b) {
if (a == 0 || b == 0) return 0;
long long gcd = compute_gcd(a, b);
return (a / gcd) * b; /* Prevents overflow prior to division */
}

int main(void) {
long long x = 144, y = 60;
long long gcd = compute_gcd(x, y);
long long lcm = compute_lcm(x, y);

printf("=== Euclidean Number Theory Solver ===\n");
printf("Inputs: %lld, %lld\n", x, y);
printf("Greatest Common Divisor (GCD): %lld\n", gcd);
printf("Least Common Multiple (LCM): %lld\n", lcm);

return 0;
}
```

Solution to Question 12: Write an ANSI C program to check whether a given integer is a Palindrome Number by reversing its digits.

```
int main(void) {
int test_vals[] = {12321, 12345, 1001, 7, -121, 8888};
int count = sizeof(test_vals) / sizeof(test_vals[0]);

printf("=== Palindromic Integer Verification ===\n");
for (int i = 0; i < count; i++) {
int v = test_vals[i];
printf("Number %8d is %s\n", v,
is_palindrome_number(v) ? "PALINDROME" : "NOT Palindrome");
}
return 0;
}
```

>>Solution to Question 14: an ANSI C program to verify whether a given positive integer is an Armstrong Number (Narcissistic Number).

```
/* 2. Compute sum of digits raised to power of digits */
temp = num;
long long sum = 0;
while (temp > 0) {
int d = temp % 10;
sum += integer_power(d, digits);
temp /= 10;
}

return (sum == (long long)num);
}

int main(void) {
int test_set[] = {153, 370, 371, 407, 1634, 54748, 123};
int count = sizeof(test_set) / sizeof(test_set[0]);

printf("=== Armstrong / Narcissistic Number Verification ===\n");
for (int i = 0; i < count; i++) {
int v = test_set[i];
printf("Number %7d: %s\n", v, is_armstrong(v) ? "ARMSTRONG" : "Regular Integer");
}
return 0;
}
```

>>Solution to Question 15: an ANSI C program to compute the Taylor series expansion of e^x to a specified convergence tolerance epsilon.

```
while (fabs(current_term) > EPSILON) {
current_term *= (x / (double)n);
sum += current_term;
n++;
}

*terms_used = n;
return sum;
}

int main(void) {
double x_vals[] = {0.5, 1.0, 2.0, -1.0};
int count = sizeof(x_vals) / sizeof(x_vals[0]);

printf("=== Maclaurin Series Approximation for e^x ===\n");
for (int i = 0; i < count; i++) {
int terms = 0;
double approx = compute_exp_taylor(x_vals[i], &terms);
double library_exp = exp(x_vals[i]);
printf("x = %+5.2f | Approx: %14.10f | math.h: %14.10f | Terms: %2d\n",
x_vals[i], approx, library_exp, terms);
}
return 0;
}
```

>>Solution to Question 16: an ANSI C program to compute the roots of a quadratic equation ax^2 + bx + c = 0, handling real, equal, and complex roots.

```
double r2 = (-b - sqrt(discriminant)) / (2.0 * a);
printf("Distinct Real Roots: Root1 = %.4f, Root2 = %.4f\n", r1, r2);
} else if (fabs(discriminant) <= 1e-9) {
double r = -b / (2.0 * a);
printf("Single Repeated Real Root: Root = %.4f\n", r);
} else {
double real_part = -b / (2.0 * a);
double imag_part = sqrt(-discriminant) / (2.0 * a);
printf("Conjugate Complex Roots: %.4f +/- %.4fi\n", real_part, imag_part);
}
}

int main(void) {
solve_quadratic(1.0, -5.0, 6.0); /* D > 0: Real distinct */
solve_quadratic(1.0, -4.0, 4.0); /* D = 0: Repeated */
solve_quadratic(1.0, 2.0, 5.0); /* D < 0: Complex roots */
return 0;
}
```

>>Solution to Question 17: an ANSI C program using switch-case to count nucleotide frequencies (A, C, G, T) and calculate GC content percentage.

```
int total_valid = count_A + count_C + count_G + count_T;
double gc_pct = 0.0;
if (total_valid > 0) {
gc_pct = ((double)(count_G + count_C) / total_valid) * 100.0;
}

printf("=== Genomic Nucleotide Profiler ===\n");
printf("Adenine (A): %5d (%5.2f%%)\n", count_A, total_valid ? (count_A*100.0/total_valid) :
0.0);
printf("Cytosine (C): %5d (%5.2f%%)\n", count_C, total_valid ? (count_C*100.0/total_valid) :
0.0);
printf("Guanine (G): %5d (%5.2f%%)\n", count_G, total_valid ? (count_G*100.0/total_valid) :
0.0);
printf("Thymine (T): %5d (%5.2f%%)\n", count_T, total_valid ? (count_T*100.0/total_valid) :
0.0);
printf("Total Bases: %5d | Invalid Chars: %d\n", total_valid, invalid);
printf("Overall GC-Content: %.2f%%\n", gc_pct);
}

int main(void) {
const char *gene_fragment = "ATGCGATCGATCGATCGATCGATCGATCGCGCATATCGATCAGCT";
analyze_dna_sequence(gene_fragment);
return 0;
}
```

>>Solution to Question 18: an ANSI C program to search a DNA sequence for Start Codon (ATG) and identify corresponding Stop Codons in the open readi

```
for (int j = i + 3; j <= len - 3; j += 3) {
if ((dna[j] == 'T' && dna[j+1] == 'A' && dna[j+2] == 'A') ||
(dna[j] == 'T' && dna[j+1] == 'A' && dna[j+2] == 'G') ||
(dna[j] == 'T' && dna[j+1] == 'G' && dna[j+2] == 'A')) {

printf(" -> In-frame STOP codon (%.3s) at position: %d (Length: %d codons)\n",
&dna[j], j + 1, (j - i) / 3);
found_stop = 1;
break; /* Terminate this ORF */
}
}
if (!found_stop) {
printf(" -> No in-frame stop codon found before sequence termination.\n");
}
}
}
}

int main(void) {
const char *genome_seq = "CCGATGAAACCCGGGTTTTAACCGATGTTTTAGCC";
scan_orfs(genome_seq);
return 0;
}
```

>>Solution to Question 19. Explain the mechanics and application of the comma operator inside a for loop header for multiple initializations and updates.

```
  int n = sizeof(arr) / sizeof(arr[0]);

  printf("Original Array: ");
  print_array(arr, n);

  /* Loop header utilizes comma operator for dual pointer tracking */
  for (int low = 0, high = n - 1; low < high; low++, high--) {
  int temp = arr[low];
  arr[low] = arr[high];
  arr[high] = temp;
  }

  printf("Reversed Array: ");
  print_array(arr, n);

  return 0;
  }
```

Solution to Question 20: Write an ANSI C program to print Pascal's Triangle up to N rows using nested for loops and combinatorial logic.

```
return 0;
}
```

```
n /= 10;
}
n = sum;
}
return n;
}

int compute_digital_root_formula(int n) {
if (n == 0) return 0;
return 1 + ((n - 1) % 9);
}

int main(void) {
int test_nums[] = {0, 5, 42, 153, 9875, 65536};
int count = sizeof(test_nums) / sizeof(test_nums[0]);

printf("=== Digital Root Calculation Benchmark ===\n");
for (int i = 0; i < count; i++) {
int val = test_nums[i];
int r_iter = compute_digital_root_iterative(val);
int r_form = compute_digital_root_formula(val);
printf("Value: %6d -> Iterative Root: %d | O(1) Formula Root: %d\n",
val, r_iter, r_form);
}
return 0;
}
```

Solution to Question 23: Write an ANSI C program to convert a decimal integer to binary, octal, and hexadecimal bases using iteration and bitwise operators.

```
  printf("Hexadecimal: 0x%08X\n", n);
  printf("Decimal: %u\n", n);
  }

  int main(void) {
  unsigned int val = 305419896; /* 0x12345678 */
  printf("=== Multi-Base Representation of Value ===\n");
  print_octal_and_hex(val);
  print_binary_32bit(val);
  return 0;
  }
```

---

hydrogen bonds (~4 deg C contribution).
- Salt-Adjusted Formula (For longer oligonucleotides > 20 bp):
Tm = 81.5 + 16.6 * log10([Na+]) + 0.41 * (%GC) - (675.0 / Length)

2. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#include <math.h>

void compute_primer_tm(const char *seq, double na_conc_molar) {
int a = 0, c = 0, g = 0, t = 0;
int len = strlen(seq);

for (int i = 0; i < len; i++) {
char base = toupper((unsigned char)seq[i]);
if (base == 'A') a++;
else if (base == 'C') c++;
else if (base == 'G') g++;
else if (base == 'T') t++;
}

int total_bp = a + c + g + t;
if (total_bp == 0) return;

/* 1. Wallace Rule */
double tm_wallace = (2.0 * (a + t)) + (4.0 * (g + c));

/* 2. Salt-Adjusted Formula */
double gc_pct = ((double)(g + c) / total_bp) * 100.0;
double tm_salt = 81.5 + (16.6 * log10(na_conc_molar)) + (0.41 * gc_pct) - (675.0 / total_bp);

printf("=== Primer Melting Temperature (Tm) Report ===\n");
printf("Sequence: %s (%d nt)\n", seq, total_bp);
printf("Composition: A=%d, C=%d, G=%d, T=%d | GC-Content: %.2f%%\n", a, c, g, t, gc_pct);
printf("Wallace Rule Estimated Tm: %.2f deg C\n", tm_wallace);
printf("Salt-Adjusted Tm ([Na+]=%.2f M): %.2f deg C\n", na_conc_molar, tm_salt);
}

int main(void) {
compute_primer_tm("ATGCGATCGATCGATCGC", 0.05); /* 18-mer primer, 50mM Na+ */
return 0;
}
```

>> Solution to Question 26

>> Solution to Question 26: Explain Duff's Device in C, detailing how interleaved switch and loop statements optimize memory block copies.

2. Loop Unrolling Rationale:
Standard byte copying:
do { *to = *from++; } while (--count > 0);
Every single byte copied incurs branch instruction overhead and loop condition decrementing.
Unrolling the loop 8 times reduces loop branch overhead by 87.5%!
Challenge: What if the total count is not an exact multiple of 8?

3. The Revolutionary Syntactical Insight:
In ANSI C, case labels inside a switch block are legal anywhere inside enclosed compound statements, EVEN INSIDE A WHILE LOOP!
Tom Duff nested a do-while loop directly across switch case labels!

4. Annotated Code Listing:

```
void send_data(register short *to, register short *from, register int count) {
register int n = (count + 7) / 8;
switch (count % 8) {
case 0: do { *to = *from++;
case 7: *to = *from++;
case 6: *to = *from++;
case 5: *to = *from++;
case 4: *to = *from++;
case 3: *to = *from++;
case 2: *to = *from++;
case 1: *to = *from++;
} while (--n > 0);
}
}
```

5. Execution Flow:
- switch (count % 8) calculates remainder (0-7) and jumps directly to the appropriate case label to copy initial leftover elements.
- Execution falls through to case 1, where the while (--n > 0) is encountered.
- If n > 0, the loop wraps back to the 'do' before case 0, subsequently executing all 8 copy operations in full rounds!

>>P Solution Statement Question 27: an ANSI C program to reverse an integer array using a while loop and pointers without auxiliary arrays.

```
void reverse_array_inplace(int *arr, int size) {
if (size <= 1) return;

int *left = arr;
int *right = arr + (size - 1);

while (left < right) {
int temp = *left;
*left = *right;
*right = temp;

left++;
right--;
}
}

void print_int_array(const int *arr, int size) {
for (int i = 0; i < size; i++) {
printf("%d ", arr[i]);
}
printf("\n");
}

int main(void) {
int data[] = {12, 45, 78, 23, 56, 89, 90};
int n = sizeof(data) / sizeof(data[0]);

printf("Array before in-place reversal: ");
print_int_array(data, n);

reverse_array_inplace(data, n);

printf("Array after in-place reversal: ");
print_int_array(data, n);

return 0;
}
```

>>Solution to Question 28: an ANSI C program to compute the factorial of an integer iteratively and analyze integer overflow boundaries.

3. Complete ANSI C Source Code:

```
#include <stdio.h>

int compute_factorial_checked(int n, unsigned long long *result) {
if (n < 0 || n > 20) {
return -1; /* Out of representable bounds for 64-bit integers */
}

unsigned long long fact = 1;
for (int i = 1; i <= n; i++) {
fact *= (unsigned long long)i;
}

*result = fact;
return 0;
}

int main(void) {
printf("=== Combinatorial Factorial Table ===\n");
for (int i = 0; i <= 22; i++) {
unsigned long long res;
if (compute_factorial_checked(i, &res) == 0) {
printf("%2d! = %20llu\n", i, res);
} else {
printf("%2d! = [OVERFLOW] Exceeds 64-bit unsigned integer capacity!\n", i);
}
}
return 0;
}
```

>> Solution to Question 29. Explain the dangling-pointer and memory leak traps that occur when managing dynamic loops and jump statements.

```
continue; /* FATAL: Skips free(data)! Memory is permanently leaked! */
}
/* ... */
free(data);
}
}
```

3. Defensive Resolution Patterns:
- Pattern 1: Allocate outside the loop if buffer size is constant, reusing the single buffer across iterations.
- Pattern 2: Ensure cleanup blocks execute prior to jump statements, or utilize a central cleanup label with goto.

>> Solution to Question 30: Write an ANSI C program to simulate a bioreactor pH and temperature feedback control loop with safety alarm thresholds.

```
  }

  /* pH Control Loop */
  if (current_ph < PH_MIN) {
  printf(" -> Corrective Action: Dispensing NaOH alkali buffer (+0.1 M).\n");
  } else if (current_ph > PH_MAX) {
  printf(" -> Corrective Action: Dispensing HCl acid buffer (-0.1 M).\n");
  } else {
  printf(" -> pH Status: Optimal physiological equilibrium.\n");
  }

  /* Temperature Control Loop */
  if (current_temp > TEMP_MAX) {
  printf(" -> Corrective Action: Activating chilled water cooling jacket.\n");
  } else if (current_temp < TEMP_MIN) {
  printf(" -> Corrective Action: Engaging auxiliary electrical heating element.\n");
  } else {
  printf(" -> Temperature Status: Nominal setpoint maintained.\n");
  }
  }

  printf("\nBioreactor monitoring cycle sequence concluded.\n");
  return 0;
  }
```
